

What is Azure DevOps?

Azure DevOps is a cloud-based DevOps platform from [Microsoft](#) that helps teams **plan, develop, test, build, deploy, and monitor software applications**.

It provides a complete set of tools to manage the **entire Software Development Life Cycle (SDLC)** from idea to production.

Think of Azure DevOps as a **single platform where developers, testers, project managers, and operations teams collaborate together**.

Example workflow:

Requirement → Development → Build → Test → Release → Deployment → Monitoring → Feedback

Without Azure DevOps, teams often use separate tools:

- Project management → Jira
- Source control → GitHub
- CI/CD → Jenkins
- Testing → TestRail
- Artifact storage → Nexus

Azure DevOps combines many of these capabilities into one ecosystem.

Why Azure DevOps?

Common challenges in software teams:

- Requirements tracked in one tool
- Code stored in another
- Manual deployments
- No automation
- Hard to track bugs
- Poor collaboration
- Release delays

- Environment inconsistencies

Azure DevOps solves these by providing:

- ✓ Centralized collaboration
 - ✓ Automation
 - ✓ Continuous Integration (CI)
 - ✓ Continuous Delivery/Deployment (CD)
 - ✓ Source control
 - ✓ Agile planning
 - ✓ Test management
 - ✓ Package management
 - ✓ Security integration
 - ✓ Reporting and dashboards
-

What is DevOps?

Before Azure DevOps, understand DevOps.

DevOps = Development + Operations

It is a culture + practices + tools that improve collaboration between development and operations teams.

Goals:

- Faster delivery
- Better quality
- Frequent releases
- Automation
- Reduced failures
- Faster recovery
- Continuous improvement

DevOps lifecycle:

1. Plan

2. Develop
3. Build
4. Test
5. Release
6. Deploy
7. Operate
8. Monitor
9. Feedback

Azure DevOps supports this entire lifecycle.

Azure DevOps Core Components

Azure DevOps consists of major services:

1. Azure Boards
2. Azure Repos
3. Azure Pipelines
4. Azure Test Plans
5. Azure Artifacts

Let's understand each in detail.

1. Azure Boards

Azure Boards is the **project planning and work tracking tool**.

Used by:

- Project Managers
- Scrum Masters
- Developers
- QA Teams

- Product Owners

Purpose:

Manage work items and Agile processes.

Features

Work Items

A work item represents a task or requirement.

Examples:

- User Story
- Bug
- Task
- Epic
- Feature
- Issue

Example:

User Story:

"As a customer, I want login functionality."

Tasks:

- Design login UI
- Create API
- Validate credentials
- Add JWT authentication
- Unit testing

Bug:

"Login button not working on mobile."

Backlogs

A prioritized list of pending work.

Example backlog:

- Payment integration
- Order history
- Email notification
- Product search

Benefits:

- Prioritization
 - Sprint planning
 - Visibility
-

Sprint Planning

Sprint = fixed development cycle (usually 2 weeks).

Example:

Sprint 1:

- Login
- Registration
- Forgot password

Sprint 2:

- Product listing
 - Search
 - Cart
-

Kanban Board

Visual workflow tracking.

Example columns:

To Do → In Progress → Testing → Done

Example:

Task:

"Create Product API"

Moves through stages.

Dashboards

Visual reporting.

Real Example

E-commerce project:

Epic:

"Shopping Module"

Features:

- Product Management
- Cart Management
- Checkout

User Stories:

- View products
- Add to cart
- Remove from cart

Tasks:

- Build API
 - Create Angular UI
 - Write unit tests
-

2. Azure Repos

Azure Repos provides source code management.

Supports:

- Git
- TFVC (Team Foundation Version Control)

Used for storing source code securely.

3. Azure Pipelines

One of the most important Azure DevOps services.

Used for automation.

Azure Pipelines automates:

- Build
- Test
- Deploy

Supports:

- .NET
- Java
- Node.js
- Python
- Angular
- React
- Docker
- Kubernetes
- Azure
- AWS
- GCP

CI/CD Concepts

Continuous Integration (CI)

Automatically build code when changes are pushed.

Flow:

Developer pushes code → Build runs → Tests execute → Feedback

Benefits:

- Early issue detection
- Faster integration
- Better code quality

Example:

Push ASP.NET Core code → build solution → run tests

Continuous Delivery (CD)

Automatically prepare deployment.

Example:

Build successful → deploy to staging

Manual approval for production

Continuous Deployment

Automatic deployment to production.

No manual approval.

Pipeline Types

Classic Pipeline

GUI-based pipeline creation.

No YAML coding needed.

Good for beginners.

YAML Pipeline

Pipeline as code.

Example:

trigger:

- main

pool:

vmImage: ubuntu-latest

steps:

- task: UseDotNet@2

inputs:

packageType: sdk

version: 8.0.x

- script: dotnet build

Benefits:

- Version controlled
 - Reusable
 - Maintainable
-

Pipeline Agents

Agents execute pipeline jobs.

Types:

Microsoft Hosted Agents

Managed by Microsoft.

Examples:

- Windows
- Ubuntu
- macOS

No setup needed.

Self Hosted Agents

Managed by your organization.

Used when:

- Custom software required
 - Internal network access needed
-

Stages

Example:

Build

Test

Deploy Dev

Deploy QA

Deploy Prod

Approvals

Production approval before deployment.

Example:

Manager approval required.

Artifacts

Build outputs stored for deployment.

Examples:

- DLL files
 - ZIP packages
 - Docker images
-

4. Azure Test Plans

Test management solution.

Used by QA teams.

Features:

- Manual testing
 - Exploratory testing
 - Test case management
 - Bug reporting
-

Components

Test Plans

Collection of test activities.

Example:

Release 1 testing plan.

Test Suites

Group test cases.

Examples:

Login Suite

Payment Suite

Test Cases

Individual tests.

Example:

Test:

"Verify valid login"

Steps:

1. Open login page
2. Enter username
3. Enter password
4. Click login
5. Verify dashboard

Expected:

Dashboard displayed.

5. Azure Artifacts

Package management service.

Store and share packages.

Supports:

- NuGet
- npm
- Maven

- Python packages

Examples:

- Internal shared library
- Common utilities package

Scenario:

Team creates:

Company.Common.Logging

Stores in Azure Artifacts.

Other teams reuse it.

Benefits:

- Reusability
 - Version control
 - Secure package hosting
-

Supporting Concepts

Service Connections

Secure connections to external services.

Examples:

- Azure subscription
- Docker Hub
- Kubernetes
- GitHub
- AWS

Used by pipelines.

Environments

Deployment targets.

Examples:

- Dev
- QA
- Production

Supports approvals/checks.

Integration Ecosystem

Azure DevOps integrates with:

- [GitHub](#)
 - Docker
 - Kubernetes
 - Jenkins
 - Slack
 - Teams
 - SonarQube
 - Terraform
-

Example Real Project Flow

ASP.NET Core + Angular e-commerce project:

Azure Boards:

Track:

- Login story
- Cart story
- Payment bug

Azure Repos:

Store:

- API code
- Angular code

Azure Pipelines:

Build API:

dotnet restore

dotnet build

dotnet test

dotnet publish

Build Angular:

npm install

npm run build

Deploy:

- Dev App Service
- QA App Service
- Production App Service

Azure Test Plans:

- Login tests
- Checkout tests

Azure Artifacts:

Store internal NuGet packages

Azure DevOps Architecture (Simple View)

Azure Boards -> Planning

Azure Repos -> Source Code

Azure Pipelines -> Automation

Azure Test Plans -> Testing

Azure Artifacts -> Package Management

Full lifecycle:

Plan → Code → Build → Test → Release → Deploy → Monitor

Licensing

Basic access includes:

- Boards
- Repos
- Pipelines (limits apply)
- Artifacts (limits apply)

Test Plans may require additional licensing.

Beginner Learning Order

Recommended order:

1. Azure DevOps overview
2. Azure Boards
3. Git + Azure Repos
4. Pipelines
5. YAML pipelines
6. CI/CD implementation
7. Release management
8. Test Plans
9. Artifacts
10. Security & permissions